🎯 Materi Interview — Project QuizYu

PENTING
Dokumen ini mencakup topik yang mungkin ditanyakan interviewer, mulai dari perkenalan diri dan arsitektur sistem hingga pemanggilan API, state management, serta debugging. Pahami alurnya, bukan sekadar menghafalnya.

Daftar Isi

1. Perkenalan Diri & Project
2. Arsitektur & Cara Kerja Aplikasi
3. State Management — Riverpod
4. Model & Data Layer
5. Cara Kerja Pemanggilan API
6. Backend — Node.js + Gemini AI
7. Flutter Teknis
8. Debugging & Error Handling
9. Responsive Design
10. Pertanyaan Jebakan & Cara Jawabnya

🎯 Materi Interview — Project QuizYu

> **PENTING**

Dokumen ini mencakup semua topik yang kemungkinan ditanya interviewer: dari perkenalan diri, arsitektur sistem, cara kerja API, state management, sampai debugging. Pelajari dan pahami alurnya, bukan sekadar dihafal.

📋 DAFTAR ISI

Perkenalan Diri & Project Arsitektur & Cara Kerja Aplikasi State Management — Riverpod Model & Data Layer Cara Kerja Pemanggilan API Backend — Node.js + Gemini AI Flutter Teknis Debugging & Error Handling Responsive Design Pertanyaan Jebakan & Cara Jawabnya

1. Perkenalan Diri & Project

"Ceritain tentang dirimu dan project yang kamu buat"

Contoh Jawaban:

"Saya [nama], developer Flutter. Saya membuat QuizYu, sebuah aplikasi kuis berbasis AI yang memungkinkan pengguna belajar topik apa saja secara interaktif. User cukup ketik topik yang ingin dipelajari, lalu AI — Gemini dari Google — akan generate soal kuis beserta penjelasannya secara otomatis. Setelah kuis selesai, ada fitur Study Guide yang merangkum materi berdasarkan soal yang salah dijawab, jadi belajarnya lebih personal dan efisien."

"Tech stack yang kamu pakai apa saja?"

Layer	Teknologi
Frontend	Flutter (Dart)
State Management	Riverpod (flutter_riverpod ^2.5.1)
HTTP Client	Dio (^5.5.0)
Render Markdown	flutter_markdown
Font	Google Fonts
Backend	Node.js + Express.js
AI Engine	Google Gemini AI (via @google/generative-ai)
Deployment	Vercel (Serverless)

"Kenapa pilih Flutter?"

"Flutter cross-platform, jadi satu codebase bisa jalan di Android, iOS, dan Web. Saya deploy QuizYu ke web via Vercel, jadi bisa diakses dari browser tanpa install app. Dart-nya juga typed sehingga lebih aman dari bug runtime dibanding JavaScript."

"Kenapa pilih Riverpod bukan Provider atau GetX?"

"Riverpod tidak bergantung pada BuildContext untuk membaca state, sehingga aman digunakan di mana saja — termasuk di luar widget tree. Provider tradisional sering error runtime jika provider tidak ditemukan di atas widget tree. Riverpod juga sudah compile-time safe. Dibanding GetX, Riverpod lebih predictable karena state-nya immutable dan mudah dibaca."

2. Arsitektur & Cara Kerja Aplikasi

"Gimana alur kerja aplikasi QuizYu dari awal sampai akhir?"

Poin kunci yang harus diingat:

MainNavigationWrapper bertindak sebagai router berbasis state — tidak ada Navigator.push manual. Navigasi antar screen terjadi otomatis bergantung pada nilai quizProvider (null → loading → data → completed). Satu ProviderScope di main.dart menjadi wadah semua state.

"Kenapa tidak pakai Navigator.push untuk pindah halaman?"

"Karena screen yang ditampilkan murni bergantung pada state dari Riverpod provider. Waktu state berubah, MainNavigationWrapper rebuild dan otomatis menampilkan widget yang sesuai. Ini pola yang lebih reaktif dan mudah di-debug karena sumber kebenaran cuma satu — quizProvider."

3. State Management — Riverpod

"Jelaskan gimana Riverpod dipakai di project ini"

Project ini punya 3 provider utama:

```
// 1. Quiz Provider – state utama sesi kuis
final quizProvider = StateNotifierProvider<QuizNotifier, AsyncValue<QuizSession?>>(
  (ref) => QuizNotifier()
);
// 2. Settings Provider – pengaturan soal (topik, jumlah, difficulty)
final settingsProvider = StateNotifierProvider<SettingsNotifier, QuizSettings>(
  (ref) => SettingsNotifier()
);
// 3. Study Guide Provider – panduan belajar hasil AI
final studyGuideProvider = StateNotifierProvider<StudyGuideNotifier, AsyncValue<String?>>(
  (ref) => StudyGuideNotifier()
);
```

Cara baca state di UI:

```
// Membaca sekaligus mendengarkan perubahan (rebuild otomatis)
final quizState = ref.watch(quizProvider);
// Membaca satu kali (tidak rebuild)
final settings = ref.read(settingsProvider);
// Memanggil method di notifier
ref.read(quizProvider.notifier).startQuiz(topic: "Flutter", count: 10, difficulty: "Sedang");
❓ "Apa itu AsyncValue dan kenapa dipakai?"
```

"AsyncValue adalah generic state dari Riverpod yang bisa menampung 3 kondisi sekaligus: data, loading, dan error. Ini sangat cocok untuk operasi async seperti hit API. Di UI kita bisa pakai .when() untuk handle ketiganya dengan clean:"

```
return quizState.when(
  data: (session) => session == null ? HomeScreen() : QuizScreen(),
  loading: () => GorgeousLoadingScreen(),
  error: (error, stack) => HomeScreen(), // Kembali ke home kalau error
);
❓ "Kenapa pakai StateNotifier bukan Notifier (generasi baru Riverpod)?"
```

"Waktu project ini dibuat, pola StateNotifier masih sangat umum dan stabil. StateNotifier cocok karena state-nya immutable — kita tidak bisa mutate langsung, harus assign state baru via state = ... Ini membuat perubahan state lebih predictable dan mudah di-trace."

"Gimana cara copyWith bekerja di sini?"

```
// Setiap kali user jawab soal, kita tidak mutate session langsung
// Kita buat object baru dengan satu field yang berubah
void answerQuestion(int optionIndex) {
  final currentSession = state.value;
  final updatedAnswers = Map<int, int>.from(currentSession.selectedAnswers);
  updatedAnswers[currentSession.currentIndex] = optionIndex;
  // copyWith = buat QuizSession baru dengan selectedAnswers yang diupdate
  state = AsyncValue.data(currentSession.copyWith(
    selectedAnswers: updatedAnswers,
  ));
}
```

"copyWith itu prinsip immutability — daripada ubah object yang ada, kita buat copy baru dengan field yang kita mau ubah. Ini penting supaya Riverpod bisa detect perubahan dan rebuild widget yang terdampak."

4. Model & Data Layer

"Jelaskan model data yang ada di project ini"

QuizQuestion — Representasi satu soal:

```
class QuizQuestion {
  final String id;
  final String question;        // Teks soal
  final List<String> options;   // 4 pilihan jawaban
  final int correctAnswerIndex; // Index jawaban benar (0-3)
  final String hint;            // Petunjuk
  final String explanation;     // Penjelasan jawaban
}
```

QuizSession — State satu sesi bermain:

```
class QuizSession {
  final List<QuizQuestion> questions;
  final int currentIndex;              // Soal ke berapa sekarang
  final Map<int, int> selectedAnswers; // soalIndex -> pilihanIndex
  final bool isCompleted;
  final String topic;
  // Computed properties:
  int get correctCount { ... }   // Hitung jawaban benar
  int get incorrectCount { ... } // Hitung jawaban salah
  int get skippedCount { ... }   // total - yang dijawab
  double get accuracy { ... }    // (benar / total) * 100
}
```

QuizSettings — Pengaturan sebelum mulai kuis:

```
class QuizSettings {
  final String topic;        // Topik yang dipilih
  final String modelVersion; // "Standard" atau "Pro"
  final int questionCount;   // Default: 10
  final String difficulty;   // "Mudah", "Sedang", "Sulit"
  final String focusArea;    // "Semua topik" / "Peluang perbaikan"
}
❓ "Kenapa selectedAnswers pakai Map<int, int> bukan List?"
```

"Karena user bisa skip soal dan kembali ke soal sebelumnya. Kalau pakai List, index dan soal harus selalu sinkron. Dengan Map, kita cukup simpan {questionIndex: selectedOptionIndex} — soal yang belum dijawab tidak punya entry di Map. Ini juga memudahkan hitung skippedCount = totalQuestions - selectedAnswers.length."

"Kenapa fromJson di QuizQuestion ada logika shuffle options?"

```
factory QuizQuestion.fromJson(Map<String, dynamic> json, String id, {bool shuffleOptions = true}) {
  final rawOptions = List<String>.from(json['options'] ?? []);
  final rawCorrectIndex = (json['correct_answer_index'] as num?)?.toInt() ?? 0;
  // Simpan teks jawaban benar dulu sebelum di-shuffle
  final String correctOptionText = rawOptions[safeIndex];
  final shuffledOptions = List<String>.from(rawOptions)..shuffle();
  // Cari lagi posisi jawaban benar setelah di-shuffle
  final newCorrectIndex = shuffledOptions.indexOf(correctOptionText);
  return QuizQuestion(options: shuffledOptions, correctAnswerIndex: newCorrectIndex, ...);
}
```

"AI kadang cenderung menaruh jawaban benar di posisi B (index 1) melulu. Jadi waktu data dari backend datang, kita acak posisi opsi di client-side. Yang penting teks jawaban benar kita track dulu, cari lagi indexnya setelah shuffle — bukan index-nya yang kita track."

5. Cara Kerja Pemanggilan API

"Gimana cara kerja pemanggilan API di QuizYu?"

Alur lengkap generate quiz:

User ketik topik → _submitQuery() di HomeScreen ↓ ref.read(quizProvider.notifier).startQuiz(topic, count, difficulty) ↓ QuizNotifier: state = AsyncValue.loading() ↓ ApiService().generateQuiz(topic, count, difficulty, modelVersion) ↓ Dio POST ke: https://quiz-yu-v2-cki8.vercel.app/api/generate-quiz Body JSON: { topic, count, difficulty, modelVersion } ↓ [Backend] Vercel menerima request → index.js ↓ [Backend] Cek API Key Gemini valid? ✅ Valid → Kirim prompt ke Google Gemini AI → dapat JSON soal ❌ Tidak valid → generateFallbackQuiz() (bank soal lokal) ↓ [Backend] Acak posisi options (randomizeQuestionsOptions) ↓ Response JSON: { is_valid, topic, questions: [...] } ↓ QuizNotifier: parse JSON → List<QuizQuestion> → QuizSession ↓ state = AsyncValue.data(session) ↓ MainNavigationWrapper rebuild → tampilkan QuizScreen

"Kenapa pakai Dio bukan package http bawaan?"

"Dio punya fitur Interceptors yang sangat berguna. Di project ini, saya pasang interceptor untuk logging setiap request dan response tanpa perlu nulis kode di setiap function. Dio juga lebih mudah untuk konfigurasi timeout, base URL, dan default headers secara global. Package http lebih sederhana tapi tidak sepowerful Dio untuk keperluan production."

```
// Interceptor logging yang dipasang satu kali, berlaku untuk semua request
_dio.interceptors.add(InterceptorsWrapper(
  onRequest: (options, handler) {
    debugPrint('[API Request] ${options.method} ${options.path}');
    return handler.next(options); // Teruskan request
  },
  onResponse: (response, handler) {
    debugPrint('[API Response] ${response.statusCode}');
    return handler.next(response); // Teruskan response
  },
  onError: (DioException e, handler) {
    debugPrint('[API Error] ${e.message}');
    return handler.next(e); // Teruskan error
  },
));
❓ "Gimana ApiService memastikan ada satu instance saja? (Singleton)"
```

```
class ApiService {
  // Static field — hanya dibuat sekali di memory
  static final ApiService _instance = ApiService._internal();
  // Factory constructor — setiap panggil ApiService() return instance yang sama
  factory ApiService() => _instance;
  // Private constructor — tidak bisa dibuat dari luar kelas
  ApiService._internal() {
    // Inisialisasi Dio di sini, hanya jalan sekali
    _dio = Dio(BaseOptions(...));
  }
}
```

"Ini pola Singleton. Tujuannya supaya konfigurasi Dio — base URL, timeout, headers, interceptors — hanya dibuat satu kali dan di-reuse. Kalau tidak singleton, tiap pemanggilan API akan membuat instance Dio baru yang membuang resource."

"Gimana cara _getBaseUrl() menentukan URL yang tepat?"

```
String _getBaseUrl() {
  if (!_useLocalServer) {
    return 'https://quiz-yu-v2-cki8.vercel.app'; // Produksi
  }
  if (kIsWeb) {
    return 'http://localhost:3000'; // Dev mode - browser
  }
  if (Platform.isAndroid) {
    return 'http://10.0.2.2:3000'; // Dev mode - Android emulator
  }
  return 'http://localhost:3000'; // Dev mode - iOS/Desktop
}
```

"Android emulator menggunakan 10.0.2.2 untuk pointing ke localhost mesin host-nya. Ini karena dari dalam emulator, localhost itu ya si emulator itu sendiri bukan laptop kita. Untuk web dan iOS, localhost langsung bisa dipakai."

"Apa itu kIsWeb dan dari mana asalnya?"

"kIsWeb adalah constant boolean dari package flutter/foundation.dart yang bernilai true jika app sedang jalan di platform web (browser), dan false di platform lain. Ini berguna untuk conditional logic yang berbeda antara web dan mobile."

6. Backend — Node.js + Gemini AI

"Gimana backend QuizYu bekerja?"

Stack backend: Node.js + Express.js, deploy ke Vercel sebagai Serverless Function.

2 endpoint utama:

Endpoint	Method	Fungsi
POST /api/generate-quiz	POST	Generate soal kuis via Gemini AI
POST /api/generate-study-guide	POST	Generate panduan belajar Markdown
GET /	GET	Health check
GET /api/health	GET	Status server

"Gimana backend berkomunikasi dengan Gemini AI?"

```
// 1. Inisialisasi client Gemini dengan API Key
const genAI = new GoogleGenerativeAI(GEMINI_API_KEY);
// 2. Pilih model (ada fallback ke model alternatif)
const modelsToTry = ['gemini-3.5-flash', 'gemini-3.1-flash-lite', 'gemini-flash-latest', 'gemini-3.8-
flash'];
for (const mName of modelsToTry) {
  const model = genAI.getGenerativeModel({
    model: mName,
    systemInstruction: systemPromptQuiz // Instruksi format JSON
  });
  const result = await model.generateContent({
    contents: [{ role: "user", parts: [{ text: promptText }] }],
    generationConfig: { responseMimeType: "application/json" }
  });
  responseText = result.response.text();
  if (responseText) break; // Kalau sukses, berhenti loop
}
❓ "Apa itu System Prompt dan kenapa penting?"
```

"System Prompt adalah instruksi awal yang menentukan 'kepribadian' dan 'aturan' AI. Di QuizYu, system prompt untuk quiz generator berisi:

Validasi input — tolak topik gibberish Format output JSON yang harus diikuti persis Aturan distribusi jawaban benar harus acak (tidak melulu di posisi B) Bahasa output harus Bahasa Indonesia

Tanpa system prompt, Gemini bisa return format bebas yang sulit di-parse oleh Flutter."

"Gimana Smart Fallback bekerja?"

Ada 2 level fallback:

Level 1 — Backend Fallback (Smart Fallback Generator):

Aktif jika: API Key Gemini tidak valid, atau semua model Gemini gagal Menggunakan bank soal hardcoded (Flutter, Kopi, Tata Surya, Sejarah) Untuk topik lain, pakai template soal generik yang diisi dinamis

Level 2 — Flutter Fallback (MockDataService):

Aktif jika: backend tidak bisa dihubungi sama sekali (server mati, CORS error, timeout) QuizNotifier catch exception dari ApiService, lalu panggil MockDataService.generateQuiz() Logic yang mirip dengan backend fallback tapi di sisi client Dart

```
// Di QuizNotifier
try {
  final result = await ApiService().generateQuiz(...); // Coba backend dulu
  // ... proses result
} catch (e, stack) {
  try {
    final mockResult = await MockDataService.generateQuiz(...); // Fallback Flutter
    // ... proses mockResult
    return;
  } catch (_) {}
  state = AsyncValue.error('Terjadi kesalahan sistem: $e', stack);
}
❓ "Gimana backend handle JSON dari Gemini yang kadang tidak bersih?"
```

```
// Gemini kadang return JSON dengan teks sebelum/sesudah-nya
// Extract bagian JSON saja dari { ... }
const firstBrace = sanitizedText.indexOf('{');
const lastBrace = sanitizedText.lastIndexOf('}');
sanitizedText = sanitizedText.substring(firstBrace, lastBrace + 1);
// Parse
try {
  quizData = JSON.parse(sanitizedText);
} catch (parseErr) {
  // Bersihkan trailing comma dan control characters
  let cleanedJSON = sanitizedText
    .replace(/,\s*([\]}])/g, '$1') // Hapus trailing comma sebelum ] atau }
    .replace(/[\u0000-\u001F]/g, ...); // Hapus control characters
  quizData = JSON.parse(cleanedJSON);
}
```

"LLM tidak selalu 100% patuh format. Kadang ada teks sebelum JSON, trailing comma, atau karakter escape yang salah. Jadi backend perlu sanitasi sebelum parse."

"Kenapa ada validasi isGibberish() di backend?"

```
// Deteksi input tidak bermakna seperti "asdfgh", "qqqqqq", dll
function isGibberish(input) {
  // 1. Terlalu pendek (< 3 karakter)
  // 2. Tidak ada huruf sama sekali
  // 3. Karakter berulang 4+ kali (aaaa, zzzz)
  // 4. Pola repetisi (asdasd, abcabc)
  // 5. Rasio vokal terlalu rendah < 15%
  // 6. Konsonan beruntun 5+ huruf
  // 7. Keyboard sliding pattern (asdf, qwer, zxcv)
}
```

"Kita mau protect API Key Gemini dari pemborosan. Daripada kirim input tidak berguna ke Gemini dan buang kuota, lebih baik validasi dulu di backend. Gemini juga melakukan validasi serupa via system prompt, tapi double-check di backend lebih hemat."

7. Flutter Teknis

"Perbedaan ConsumerWidget dan ConsumerStatefulWidget?"

ConsumerWidget	ConsumerStatefulWidget	
Kapan dipakai	Widget sederhana yang hanya butuh membaca state Riverpod	Widget yang butuh state lokal (TextEditingController, AnimationController, FocusNode) DAN state Riverpod
Contoh di project	MainNavigationWrapper, QuizScreen, DashboardScreen	HomeScreen (punya _searchController dan _focusNode)
ref tersedia di	build(context, ref)	ref property dari ConsumerState

"Gimana cara Hint Accordion bekerja?"

```
class HintAccordion extends StatefulWidget { ... }
class _HintAccordionState extends State<HintAccordion> {
  bool _isExpanded = false; // State lokal - tidak perlu Riverpod
  @override
  Widget build(BuildContext context) {
    return AnimatedCrossFade(
      firstChild: SizedBox.shrink(),   // Tertutup
      secondChild: Text(widget.hintText), // Terbuka
      crossFadeState: _isExpanded
          ? CrossFadeState.showSecond
          : CrossFadeState.showFirst,
      duration: Duration(milliseconds: 250),
    );
  }
}
```

"HintAccordion pakai StatefulWidget biasa karena state-nya (buka/tutup) bersifat lokal dan tidak perlu dishare ke widget lain. Riverpod overkill untuk kasus ini."

"Apa itu ref.listen dan bedanya dengan ref.watch?"

```
// ref.watch — rebuild widget setiap state berubah (dipakai di build)
final quizState = ref.watch(quizProvider);
// ref.listen — jalankan side effect tanpa rebuild widget (dipakai di build juga, tapi tidak rebuild)
ref.listen<AsyncValue>(quizProvider, (previous, next) {
  next.whenOrNull(
    error: (error, stackTrace) {
      ScaffoldMessenger.of(context).showSnackBar(...); // Side effect: tampilkan snackbar
    },
  );
});
```

"ref.watch dipakai untuk membaca state yang memengaruhi tampilan. ref.listen dipakai untuk side effect seperti navigasi atau show snackbar yang tidak seharusnya dipanggil di tengah-tengah build. Kalau kita pakai ref.watch untuk snackbar, bisa terjadi showSnackBar dipanggil saat sedang build dan crash."

"Kenapa dispose() penting di HomeScreen?"

```
@override
void dispose() {
  _searchController.dispose(); // Bebaskan resource TextField
  _focusNode.dispose();         // Bebaskan resource FocusNode
  super.dispose();
}
```

"Kalau tidak di-dispose, TextEditingController dan FocusNode akan tetap di memory meskipun widget sudah dibuang dari tree — itu memory leak. Flutter bahkan akan kasih warning di debug console kalau kita lupa dispose controller."

"Gimana _submitQuery parse jumlah soal dari teks?"

```
void _submitQuery(String query) {
  // Contoh query: "Python Dasar 10 soal"
  // User bisa ketik jumlah soal langsung di input
  final numMatch1 = RegExp(r'(\d+)\s*(?:soal|pertanyaan|items?|buah)', caseSensitive:
false).firstMatch(query);
  // "10 soal" → group(1) = "10"
  final numMatch3 = RegExp(r'\b(\d+)\b').firstMatch(query);
  // Fallback: ambil angka pertama yang ditemukan
  final String? foundNum = numMatch1?.group(1) ?? numMatch2?.group(1) ?? numMatch3?.group(1);
  if (foundNum != null) {
    final parsed = int.tryParse(foundNum);
    if (parsed != null && parsed >= 1 && parsed <= 30) {
      count = parsed; // Override default dari settings
    }
  }
}
8. Debugging & Error Handling
❓ "Gimana cara debug aplikasi ini?"
```

Tools debugging:

Flutter DevTools — Inspect widget tree, memory, performance Dio Interceptor — Log semua request/response di debug console debugPrint — Log yang hanya muncul di debug mode, tidak di production Riverpod DevTools — Inspect state semua provider

Log pola yang dipakai di project ini:

```
debugPrint('🌐 [LOG TIDAK ADA KONEKSI INTERNET] $displayMsg');
debugPrint('⏳ [LOG BATAS KUOTA AI GEMINI TERLAMPAUI] $displayMsg');
debugPrint('❌ [LOG KESALAHAN SISTEM] $displayMsg');
```

"Saya pakai emoji prefix biar mudah dicari di console. Tinggal Ctrl+F cari 🌐 atau ⏳ langsung ketemu."

"Gimana cara handle error koneksi internet?"

```
String _parseDioError(DioException e) {
  // Cek tipe error spesifik dari Dio
  if (e.type == DioExceptionType.connectionError ||
      e.type == DioExceptionType.connectionTimeout ||
      e.error?.toString().contains('SocketException') == true) {
    return '[OFFLINE] Tidak ada koneksi internet...';
  }
  // Cek error 429 (Rate Limit Gemini)
  if (e.response?.data is Map) {
    final data = e.response!.data as Map;
    if (data['error_message']?.contains('429') == true) {
      return '[QUOTA_EXCEEDED] Batas Kuota AI Gemini Terlampaui...';
    }
  }
  // Error lainnya
  return '[SYSTEM_ERROR] $cleanMsg';
}
❓ "Gimana UI menampilkan error yang berbeda-beda?"
```

```
// Di HomeScreen dan MainNavigationWrapper — bedakan 3 jenis error
final isOffline = rawMsg.contains('[OFFLINE]');
final isQuota = !isOffline && rawMsg.contains('[QUOTA_EXCEEDED]');
ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: Icon(
      isOffline ? Icons.wifi_off_rounded        // 📶 Offline
      : isQuota ? Icons.hourglass_top_rounded   // ⏳ Quota
      : Icons.error_outline_rounded,            // ❌ Error lain
    ),
    backgroundColor: isOffline
      ? Color(0xFFD97706)   // Orange - offline
      : isQuota
      ? Color(0xFF7F5AF0)   // Purple - quota
      : theme.colorScheme.error, // Red - error umum
  )
);
❓ "Apa yang terjadi kalau backend return is_valid: false?"
```

```
// Di QuizNotifier.startQuiz()
if (result['is_valid'] == false) {
  final errorMsg = result['error_message'] ?? 'Gagal membuat kuis.';
  state = AsyncValue.error(errorMsg, StackTrace.current);
  return; // Berhenti proses, tidak lanjut ke MockData
}
```

"Kalau is_valid false, artinya topik yang diinput tidak valid (gibberish). Kita set state ke error dengan pesan dari backend, lalu UI menampilkan snackbar. Kita tidak fallback ke MockData karena validasi topik harus tetap strict — tidak mau generate soal dari topik tidak bermakna."

"Gimana cara debug Riverpod state?"

```
// Cara 1: Print di notifier
void answerQuestion(int optionIndex) {
  debugPrint('Current state before answer: ${state.value?.currentIndex}');
  // ...
  debugPrint('State after answer: ${state.value?.selectedAnswers}');
}
// Cara 2: Listen di widget
ref.listen(quizProvider, (prev, next) {
  debugPrint('State changed: $prev -> $next');
});
// Cara 3: Flutter DevTools > Riverpod tab (dengan riverpod_lint)
❓ "Apa yang terjadi kalau currentQuestion null di QuizScreen?"
```

```
final session = quizState.value;
if (session == null) {
  return Scaffold(body: Center(child: CircularProgressIndicator()));
}
final currentQuestion = session.currentQuestion;
if (currentQuestion == null) {
  return Scaffold(body: Center(child: CircularProgressIndicator())); // Guard clause
}
```

"Guard clause untuk null safety. Meskipun secara logika session tidak null ketika QuizScreen ditampilkan (karena MainNavigationWrapper sudah filter), kita tetap guard untuk keamanan. Ini menghindari null pointer

exception yang bisa crash app."

9. Responsive Design

"Gimana aplikasi ini handle berbagai ukuran layar?"

```
// Breakpoints yang konsisten di semua screen
final bool isMobile = screenWidth < 600;
final bool isTablet = screenWidth >= 600 && screenWidth < 1024;
final bool isDesktop = screenWidth >= 1024;
// Contoh penggunaan: ukuran font responsif
final double titleFontSize = isDesktop ? 40.0 : (isTablet ? 34.0 : 28.0);
// Contoh penggunaan: max width container agar tidak terlalu lebar di desktop
final double contentMaxWidth = isDesktop ? 900 : isTablet ? 760 : double.infinity;
// ConstrainedBox untuk limit lebar
Center(
  child: ConstrainedBox(
    constraints: BoxConstraints(maxWidth: contentMaxWidth),
    child: ... // Konten
  ),
)
```

"Aplikasi ini juga bisa jalan di web browser desktop. Kalau tidak diberi maxWidth, konten akan stretch full width dan terlihat aneh. ConstrainedBox memastikan konten tetap terbatas di area yang mudah dibaca."

"Kenapa pakai LayoutBuilder bukan MediaQuery langsung?"

"LayoutBuilder lebih tepat karena memberi tahu constraint dari parent widget, bukan ukuran layar keseluruhan. Ini penting kalau widget ada di dalam Column, Row, atau container lain yang ukurannya berbeda dengan ukuran layar. MediaQuery.of(context).size.width cocok di level top widget, tapi LayoutBuilder lebih fleksibel di level widget yang lebih dalam."

10. Pertanyaan Jebakan & Cara Jawabnya

"Kalau ada 1000 soal di satu session, performa gimana?"

"Sebenarnya QuizYu dibatasi max 30 soal per session (divalidasi di backend maupun frontend). Tapi kalau memang ada banyak data, yang perlu diperhatikan adalah ListView.builder vs Column. Saat ini soal direnderer satu per satu (bukan semua sekaligus), dan pilihan jawaban di-generate dengan List.generate yang bersifat lazy. Untuk scale lebih besar, bisa pakai ListView.builder agar hanya render yang visible."

"Kenapa tidak pakai local storage / SQLite untuk simpan history?"

"Scope MVP project ini fokus pada core experience: buat kuis → kerjakan → lihat hasil. Persistent storage adalah enhancement yang bisa ditambahkan. Kalau mau tambahkan, bisa pakai shared_preferences untuk simpan skor atau sqflite/drift untuk history session lengkap. Saat ini state hilang ketika app di-restart, dan itu sesuai dengan scope awal."

"Gimana kalau user tekan back button di QuizScreen?"

"Di Android, back button akan trigger sistem navigation. Karena QuizYu tidak pakai Navigator.push tapi routing berbasis state, tidak ada route untuk 'pop back'. Yang ada adalah dialog konfirmasi exit yang manual saya tambahkan di QuizScreen via _showExitConfirmation(). Kalau user konfirmasi keluar, resetQuiz() dipanggil yang reset state ke null, dan MainNavigationWrapper otomatis kembali ke HomeScreen."

"Kenapa tidak pakai setState saja, kenapa perlu Riverpod?"

"setState itu state lokal — hanya berlaku di satu widget. Masalahnya, state QuizSession harus dishare antara banyak widget: HomeScreen (untuk trigger start), QuizScreen (untuk tampilkan soal), DashboardScreen (untuk tampilkan hasil), dan StudyGuideScreen (untuk ambil data session yang sudah selesai). Kalau pakai setState, kita harus meneruskan data melalui banyak widget data ini dari widget ke widget, yang sangat tidak scalable. Riverpod mengatasi ini dengan global state yang bisa diakses dari mana saja."

"Apa kelemahan dari aplikasi ini?"

Jujur saja, ini justru menunjukkan kamu mature:

"Beberapa hal yang bisa ditingkatkan:

Tidak ada offline cache — Soal yang sudah di-generate tidak disimpan. Kalau app di-restart, harus generate ulang. Tidak ada autentikasi — Semua user pakai API yang sama, tidak ada personalisasi data per user. Rate limit tidak ada di level app — User bisa spam tombol generate dan buang kuota Gemini. Test coverage masih minimal — Widget test baru ada basic scaffold test. Idealnya ada unit test untuk QuizNotifier dan integration test untuk full flow.*" ⚡ Cheat Sheet — Jawaban Singkat

Pertanyaan	Jawaban Singkat
Navigasi pakai apa?	State-based routing via MainNavigationWrapper + quizProvider
HTTP client?	Dio dengan Interceptor untuk logging
State management?	Riverpod — StateNotifierProvider + AsyncValue
Singleton ApiService?	Private constructor + factory constructor + static field
Handle offline?	DioExceptionType.connectionError → return pesan [OFFLINE]
Fallback kalau Gemini down?	Backend → generateFallbackQuiz() (bank soal lokal)
Fallback kalau backend down?	Flutter → MockDataService.generateQuiz()
Shuffle jawaban kenapa?	AI cenderung taruh jawaban benar di posisi B, shuffle untuk fairness
copyWith kenapa?	Immutability — buat object baru daripada mutate langsung
ref.watch vs ref.listen?	watch = rebuild widget, listen = side effect tanpa rebuild

> **TIPS**

Tips saat interview: Saat menjelaskan, sambil gambar flowchart di whiteboard atau kertas. Visualisasi alur data (User → HomeScreen → QuizNotifier → ApiService → Backend → Gemini → kembali ke Flutter) sangat membantu interviewer menilai pemahaman arsitektur kamu.